```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from sklearn.impute import SimpleImputer,KNNImputer
from sklearn.preprocessing import LabelEncoder,StandardScaler
```

New York Yellow Cab Data

- Yellow taxi trip records include fields capturing pickup and drop-off dates/times, pickup and drop-off locations, trip distances, itemized fares, rate types, payment types, and driver-reported passenger counts.
- The data used in this dataset was collected and provided to the NYC Taxi and Limousine Commission (TLC) by technology providers authorized under the Taxicab & Livery Passenger Enhancement Programs (TPEP/LPEP).
    - tpep_pickup_datetime : Time at which the passenger was picked up
    - tpep_dropoff_datetime : Time at which the passenger was dropped off at the requested destination
    - passenger count : Number of passengers picked up
    - trip_distance : Distance of the trip
    - store_and_fwd_flag : Indicates whether the specific trip record was held in the vehicle's local memory (stored) before being sent to the vendor.
    - PULocationID : Pickup location ID
    - DOLocationID : Drop off Location ID
    - payment_type : It is a numeric code that indicates how the passenger paid for their ride.
        0. Flex Flare trip
        1. Credit Card
        2. Cash
        3. No Charge
        4. Dispute
        5. Unknown
        6. Voided Trip
    - fare_amount : The fare_amount field represents the time-and-distance fare calculated by the taximeter.
    - extra : "extra" refers specifically to miscellaneous charges and surcharges.
    - mta_tax : represents a mandatory tax applied to most trips.
    - tip_amount : Amount recieved through tips
    - tolls_amount : Represents the total dollar value of all toll charges paid by the passenger during a specific trip.
    - improvement_surcharge : The improvement surcharge in NYC yellow taxi data is a mandatory fee, Implemented to fund the Metropolitan Transportation Authority (MTA).
    - congestion_surcharge : The congestion surcharge in NYC yellow taxi data is a mandatory fee of $2.50 per ride$(0.75 for shared rides) applied to trips that start, end, or pass through Manhattan below 96th Street.
    - Airport_Fee : epresents a specific surcharge applied to trips that pick up or drop off passengers at designated New York City airports (typically JFK, LGA, or EWR).
    - cbd_congestion_fee : Represents the Congestion Relief Zone Toll (or congestion surcharge) imposed by New York State on for-hire transportation trips.

Importing Data

```
yellow_cab = pd.read_parquet('/content/drive/MyDrive/DM/datasets/yellow_tripdata_2025-01.parquet')
```

Setting the index column

```
yellow_cab.index = yellow_cab.VendorID
yellow_cab = yellow_cab.drop(columns=['VendorID'])
```

Converting the dropoff and pickup time fields to datetime objects

```
yellow_cab['tpep_dropoff_datetime'] = pd.to_datetime(yellow_cab['tpep_dropoff_datetime'])
yellow_cab['tpep_pickup_datetime'] = pd.to_datetime(yellow_cab['tpep_pickup_datetime'])
yellow_cab.head()
```

VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_f
1	2025-01-01 00:18:38	2025-01-01 00:26:59	1.0	1.60	1.0	
1	2025-01-01 00:32:40	2025-01-01 00:35:13	1.0	0.50	1.0	
1	2025-01-01 00:44:04	2025-01-01 00:46:01	1.0	0.60	1.0	
2	2025-01-01 00:14:27	2025-01-01 00:20:01	3.0	0.52	1.0	
2	2025-01-01 00:21:34	2025-01-01 00:25:06	3.0	0.66	1.0	

Information about dataset

Double-click (or enter) to edit

Shape of the dataset

```
print(f"Number of instances: {yellow_cab.shape[0]}")
print(f"Number of features: {yellow_cab.shape[1]}")
```

```
Number of instances: 3475226
Number of features: 19
```

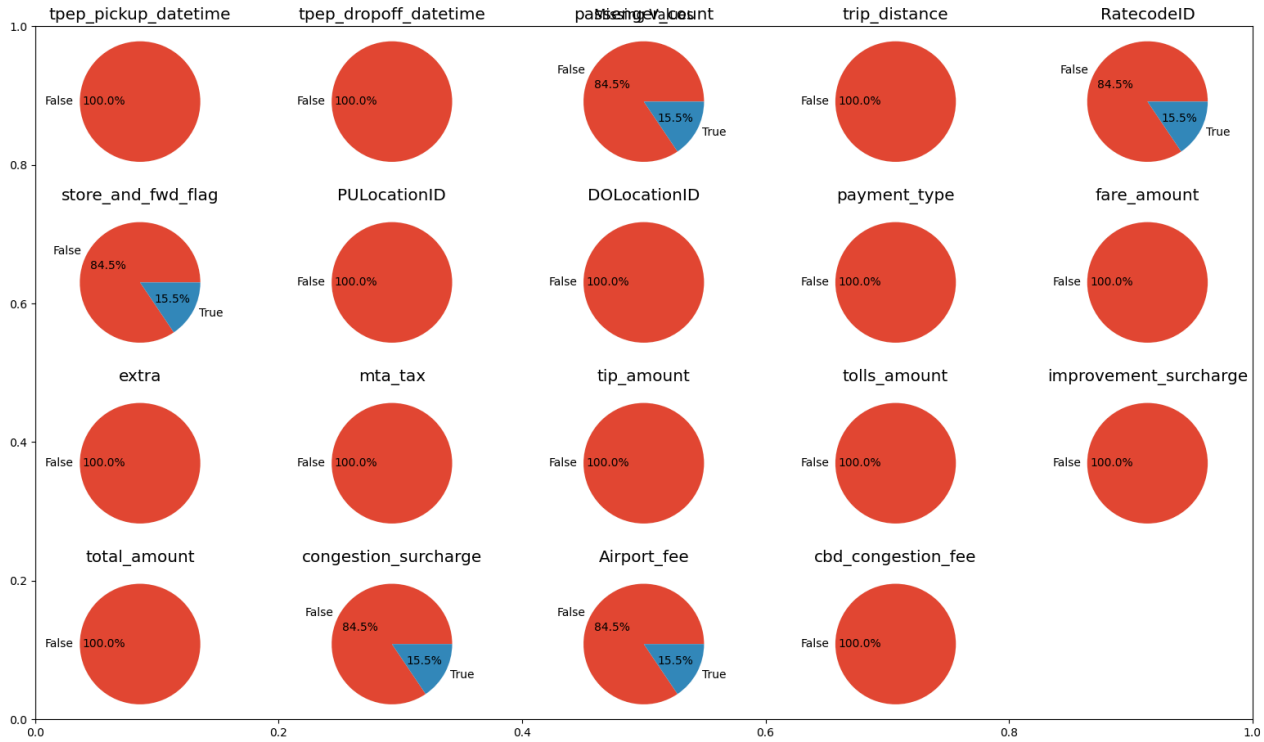
Number of non-null values in each field

```
yellow_cab.info(show_counts=True)
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 3475226 entries, 1 to 2
Data columns (total 19 columns):
 #   Column                 Non-Null Count    Dtype
---  ------                 --------------    -----
 0   tpep_pickup_datetime   3475226 non-null  datetime64[us]
 1   tpep_dropoff_datetime  3475226 non-null  datetime64[us]
 2   passenger_count        2935077 non-null  float64
 3   trip_distance          3475226 non-null  float64
 4   RatecodeID             2935077 non-null  float64
 5   store_and_fwd_flag     2935077 non-null  object
 6   PULocationID           3475226 non-null  int32
 7   DOLocationID           3475226 non-null  int32
 8   payment_type           3475226 non-null  int64
 9   fare_amount            3475226 non-null  float64
 10  extra                  3475226 non-null  float64
 11  mta_tax                3475226 non-null  float64
 12  tip_amount             3475226 non-null  float64
 13  tolls_amount           3475226 non-null  float64
 14  improvement_surcharge  3475226 non-null  float64
 15  total_amount           3475226 non-null  float64
 16  congestion_surcharge   2935077 non-null  float64
 17  Airport_fee            2935077 non-null  float64
 18  cbd_congestion_fee     3475226 non-null  float64
dtypes: datetime64[us](2), float64(13), int32(2), int64(1), object(1)
memory usage: 490.5+ MB
```

Visualization of null values

```
plt.figure(figsize=(19.2,11))
plt.title('Missing Values')
plt.style.use("ggplot")
for col in yellow_cab:
  col_index = yellow_cab.columns.get_loc(col)
  plt.subplot(4,5,col_index+1)
  plt.pie(yellow_cab[col].isna().value_counts(dropna=False),labels=yellow_cab[col].isna().value_counts(dropna=Fa
  plt.title(col)
plt.show()
```



Columns with null values

```
def get_na_cols():
  cols = list()
  for col in yellow_cab:
    if(yellow_cab[col].isna().sum() > 0):
      cols.append(col)
  return cols

na_cols = get_na_cols()
na_cols
```

```
['passenger_count',
 'RatecodeID',
 'store_and_fwd_flag',
 'congestion_surcharge',
 'Airport_fee']
```

Binning trip distance into 4 categories

- short
- small
- long
- large

```
bins = [0,0.98,1.67,3.1,276424]
yellow_cab['trip_distance'] = pd.cut(yellow_cab.trip_distance,bins,labels=['short','small','long','large'])

encoder = LabelEncoder()
yellow_cab['trip_distance'] = encoder.fit_transform(yellow_cab['trip_distance'])
classes = encoder.classes_
classes
```

```
array(['large', 'long', 'short', 'small', nan], dtype=object)
```

Imputing missing values

Median strategy for passenger_count

```
simple = SimpleImputer(strategy='median')
simple_fit = simple.fit(yellow_cab[['passenger_count']])
yellow_cab['passenger_count'] = simple_fit.transform(yellow_cab[['passenger_count']])
```

Median strategy for RatecodeID

```
simple = SimpleImputer(strategy='median')
simple_fit = simple.fit(yellow_cab[['RatecodeID']])
yellow_cab['RatecodeID'] = simple_fit.transform(yellow_cab[['RatecodeID']])
```

most_frequent for store_and_fwd_flag

```
encoder = LabelEncoder()
yellow_cab['store_and_fwd_flag'] = encoder.fit_transform(yellow_cab['store_and_fwd_flag'])
classes = encoder.classes_
classes
```

```
array(['N', 'Y', None], dtype=object)
```

```
yellow_cab['store_and_fwd_flag'] = yellow_cab['store_and_fwd_flag'].replace(2,np.nan)
```

```
simple = SimpleImputer(strategy='most_frequent')
simple_fit = simple.fit(yellow_cab[['store_and_fwd_flag']])
yellow_cab['store_and_fwd_flag'] = simple_fit.transform(yellow_cab[['store_and_fwd_flag']])
```

Mean Strategy for congestion_surcharge

```
simple = SimpleImputer(strategy='mean')
simple_fit = simple.fit(yellow_cab[['congestion_surcharge']])
yellow_cab['congestion_surcharge'] = simple_fit.transform(yellow_cab[['congestion_surcharge']])
```

Mean strategy for Airport_fee

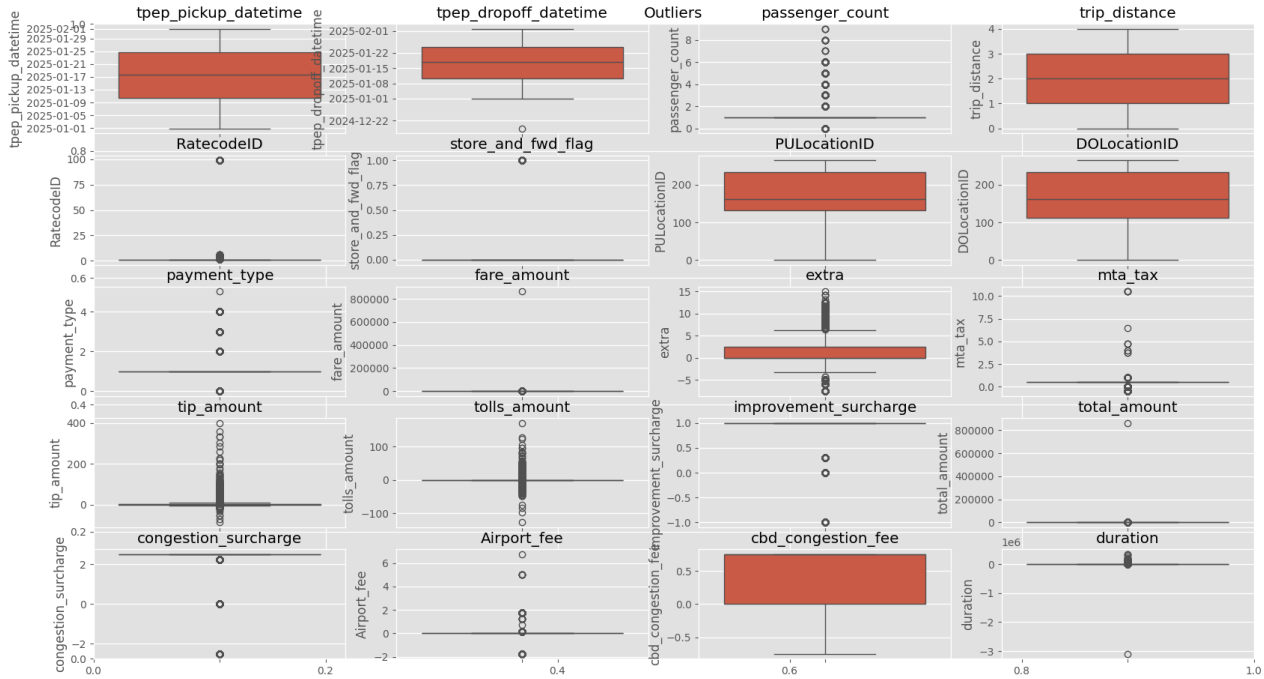
```
simple = SimpleImputer(strategy='mean')
simple_fit = simple.fit(yellow_cab[['Airport_fee']])
yellow_cab['Airport_fee'] = simple_fit.transform(yellow_cab[['Airport_fee']])
```

Reducing the number of features

```
yellow_cab['duration'] = yellow_cab['tpep_dropoff_datetime'] - yellow_cab['tpep_pickup_datetime']
yellow_cab['duration'] = yellow_cab['duration'].dt.total_seconds()
```

Checking outliers by plotting boxplots

```
plt.figure(figsize=(20,11))
plt.title('Outliers')
plt.style.use("ggplot")
for col in yellow_cab:
  col_index = yellow_cab.columns.get_loc(col)
  plt.subplot(5,4,col_index+1)
  sns.boxplot(yellow_cab[col])
  plt.title(col)
plt.show()
```



Normalization

```
scaler = StandardScaler()
yellow_cab_transformed = scaler.fit_transform(yellow_cab.iloc[:,2:])
yellow_cab = pd.DataFrame(yellow_cab_transformed,columns=yellow_cab.columns[2:])
```

```
yellow_cab.head()
```

	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type	fare_a
0	-0.360234	1.222214	-0.116975	-0.046957	0.988826	1.050044	-0.052219	-0.
1	-0.360234	0.377099	-0.116975	-0.046957	1.097304	1.050044	-0.052219	-0.
2	-0.360234	0.377099	-0.116975	-0.046957	-0.374892	-0.333208	-0.052219	-0.
3	2.503730	0.377099	-0.116975	-0.046957	1.221278	1.150906	1.373637	-0.
4	2.503730	0.377099	-0.116975	-0.046957	1.221278	-0.693430	1.373637	-0.